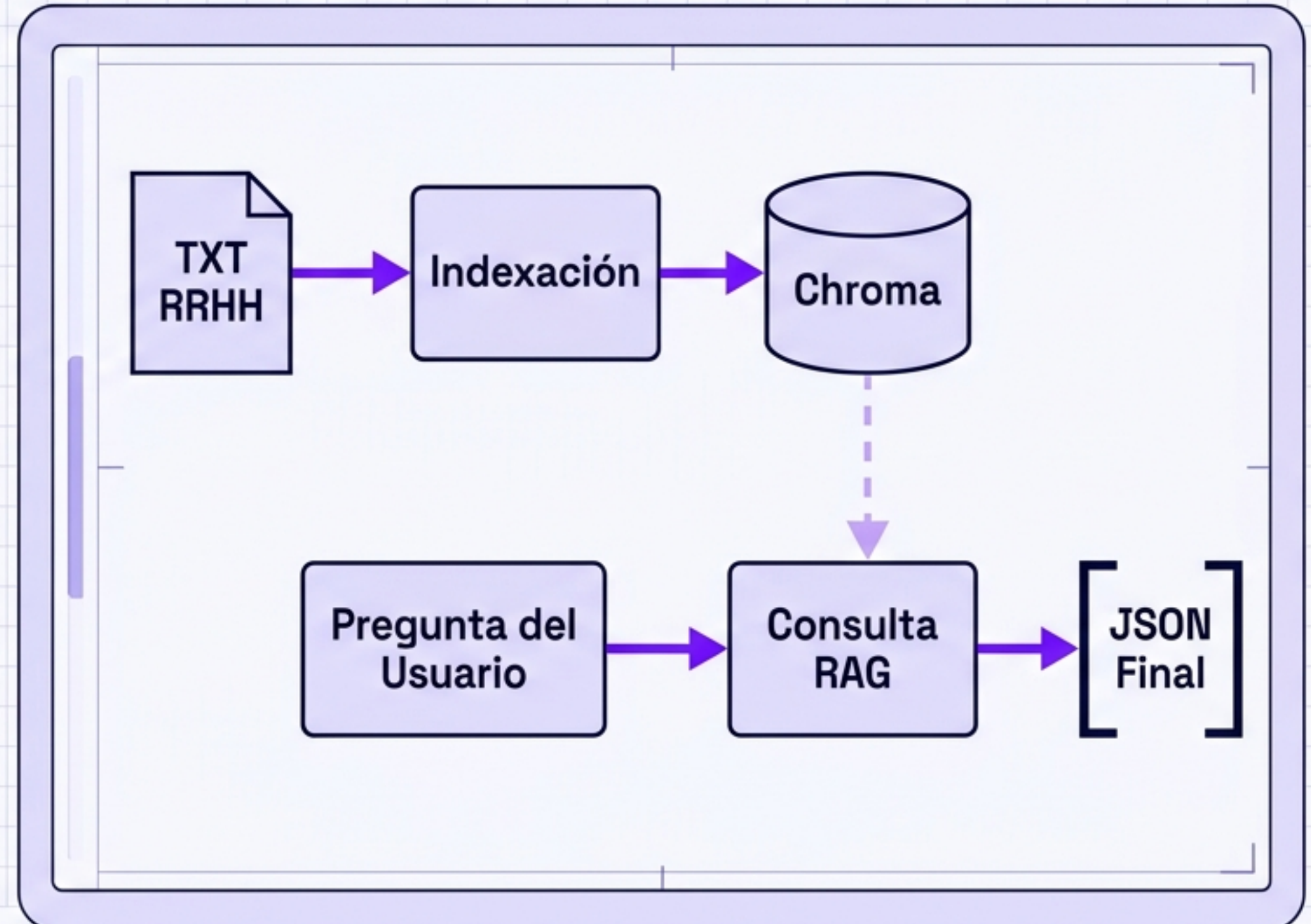


Proyecto Integrador RAG FAQ

Construyendo un Chatbot de Soporte con LangChain + Chroma

- **Objetivo:** Chatbot de soporte para SaaS de Recursos Humanos.
- **Regla de Oro:** Responder exclusivamente usando el contexto recuperado. Prohibido inventar.
- **Contrato de Salida:** JSON estricto con 3 claves exactas.
- **Stack Base:** Python, LangChain, Chroma, OpenAI.

Error Común: Pensar que LangChain reemplaza la lógica. Python es el pegamento; LangChain solo da las piezas.



Paso 1 y 2: Repositorio y Dependencias

```
Repo/  
├── venv/  
├── src/  
├── data/  
├── requirements.txt  
└── .env.example
```

Teoría Mínima: Un entorno virtual aísla dependencias para evitar conflictos.

Rúbrica: Reproducibilidad (README y requirements).

```
python -m venv venv  
  
source venv/bin/activate  
  
pip install langchain chromadb  
openai python-dotenv pytest  
  
pip freeze > requirements.txt
```

Resultado Esperado: Carpeta venv/ creada y requirements.txt listo.

Paso 3 y 4: Gestión de Secretos y Conocimiento

Local - .env



```
OPENAI_API_KEY=sk-tu_clave_real_secreta
```

¡No subir a GitHub! Excluido vía .gitignore

Público - .env.example



```
OPENAI_API_KEY=tu_clave_aqui  
CHUNK_SIZE=500  
TOP_K=4
```

Plantilla segura para compartir.

data/faq_document.txt

Políticas de RRHH: Las vacaciones deben solicitarse con 15 días de anticipación...

Teoría Mínima: El RAG depende de la calidad del TXT. Mínimo 1000 palabras.

Paso 5: Preparando utils.py

```
def clean_text(text: str) -> str:  
    return ' '.join(text.split())
```

Previene chunks sucios.
Elimina saltos de línea y
espacios extra del TXT
original.

```
def validate_question(question: str) -> bool:  
    return bool(question and question.strip())
```

Evita consultas vacías que
harían fallar al modelo de
embeddings (Garbage in,
Garbage out).

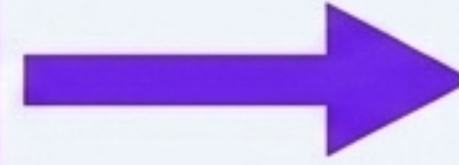
Regla de Diseño

Modularizar evita la repetición. Usamos Python puro para la lógica de control antes de depender de LangChain.

Paso 6 (A): Pipeline de Indexación - Lectura



**TextLoader
(LangChain)**



**Clean Text
(utils.py)**

```
from langchain_community.document_loaders import TextLoader
from utils import clean_text

loader = TextLoader('data/faq_document.txt', encoding='utf-8')
docs = loader.load()
docs[0].page_content = clean_text(docs[0].page_content)
```

page_content

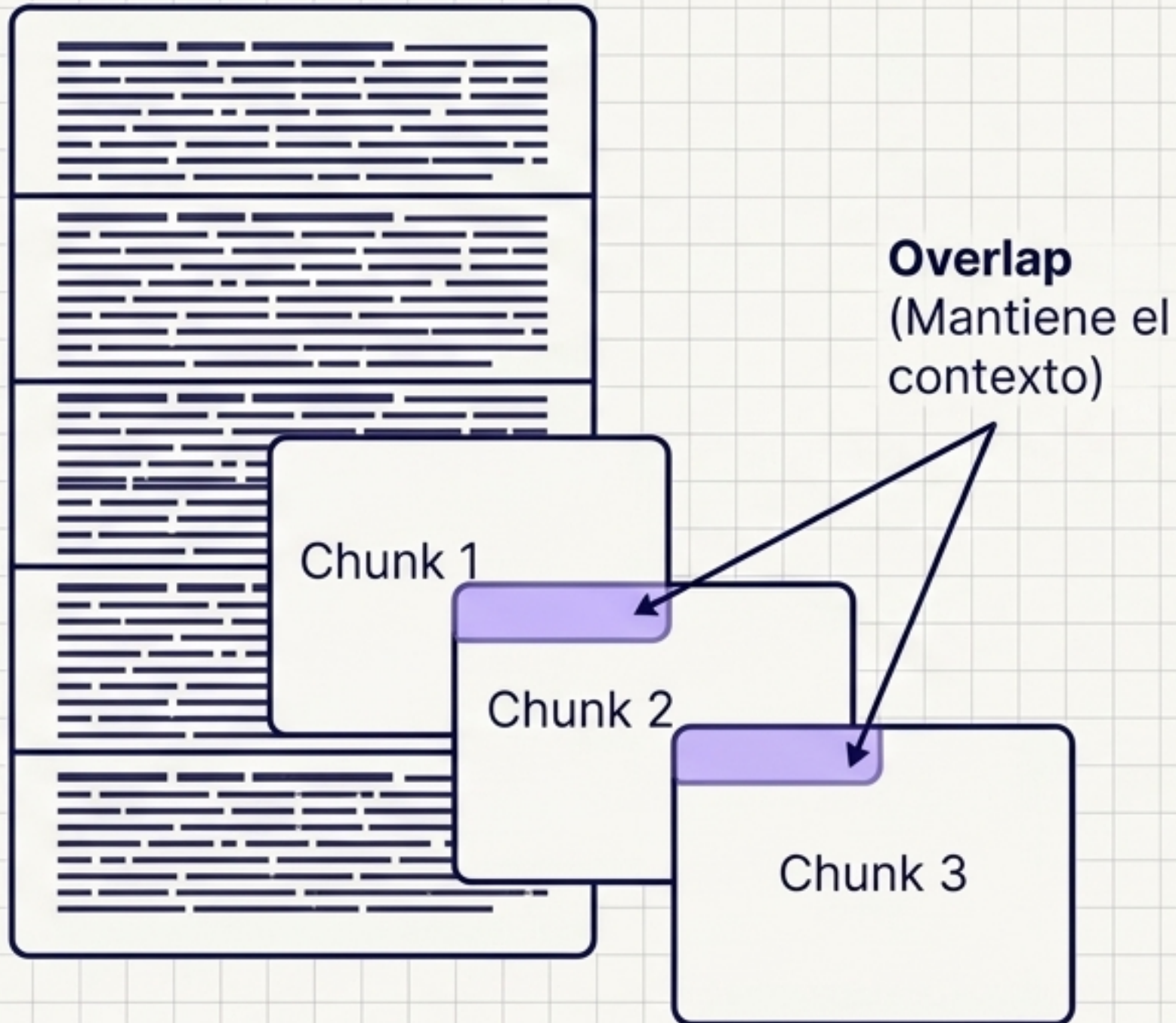
[Texto limpio]

metadata

{source: 'faq_document.txt'}

Teoría Mínima: Un Document guarda contenido y metadata. Sin metadata, perdemos la trazabilidad de la evidencia.

Paso 6 (B): Pipeline de Indexación - Fragmentación



```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = text_splitter.split_documents(docs)
print(f'Generados {len(chunks)} chunks.')
```

Rúbrica del PI: Generar 20 chunks o más.

Error Común: Usar cero overlap corta ideas por la mitad y el modelo pierde el hilo conductor.

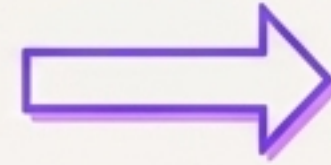
Paso 6 (C): Embeddings y Persistencia Chroma



Chunks de Texto


$$\begin{bmatrix} [0.12, 0.55, -0.9\dots] \\ [0.12, 0.55, -0.9\dots] \\ [0.12, 0.55, -0.9\dots] \end{bmatrix}$$

OpenAI Embeddings



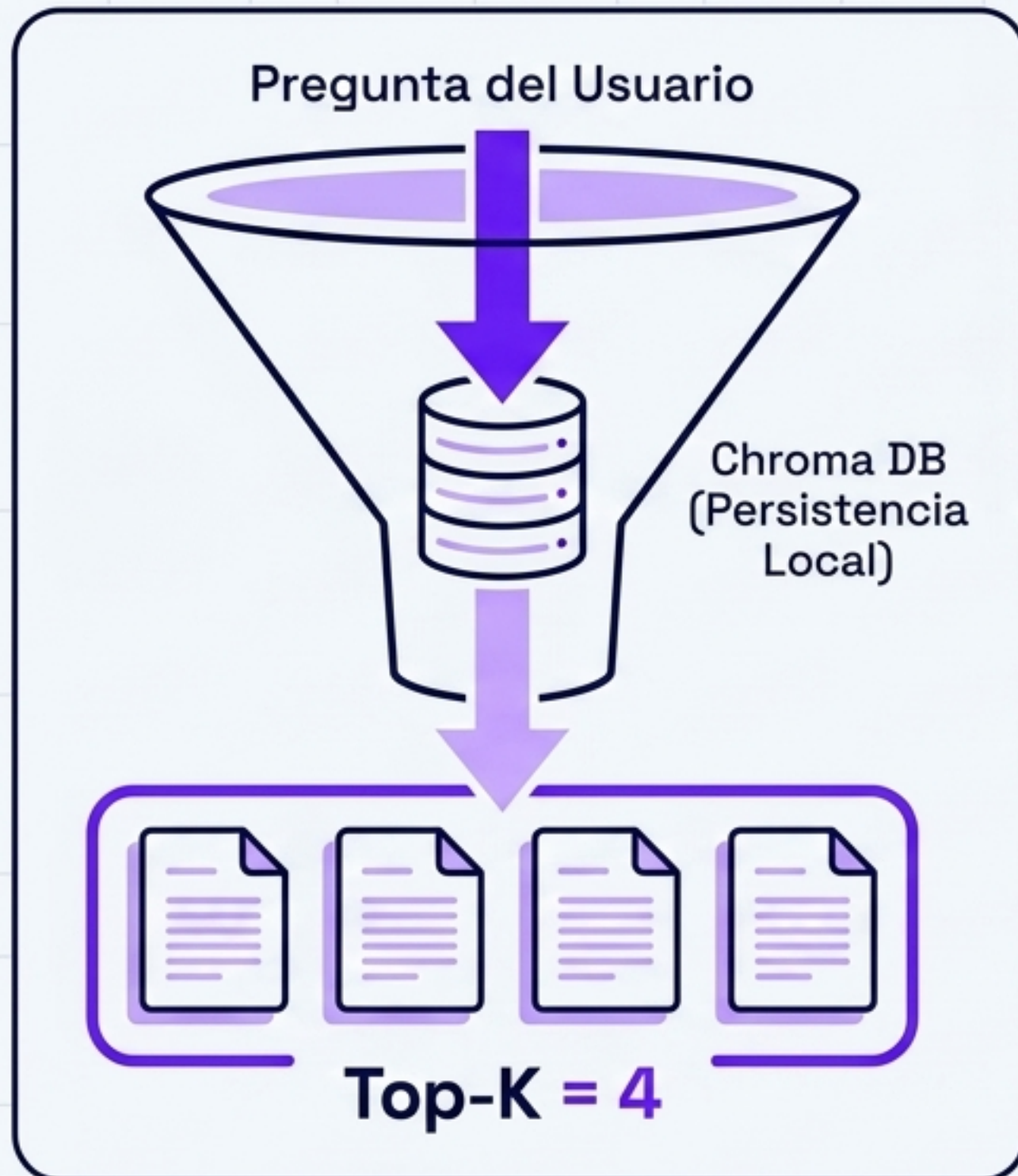
Chroma DB
(Persistencia Local)

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory='storage/chroma'
)
```

Teoría Mínima: El embedding traduce texto a significado semántico. Chroma guarda estos vectores en el disco duro (storage/chroma).

Paso 7 (A): Pipeline de Consulta - Recuperación



```
vectorstore = Chroma(  
    persist_directory='storage/chroma',  
    embedding_function=OpenAIEmbeddings()  
)  
  
retriever = vectorstore.as_retriever(search_kwargs={'k': 4})  
  
docs = retriever.invoke('¿Cómo pido vacaciones?')
```

Teoría Mínima:

El retriever es el corazón del debug. Si el bot responde basura, revisen qué recuperó. Top-K=4 cumple la rúbrica (2 a 5 chunks) y provee evidencia sin saturar el contexto.

Paso 7 (B): El Contrato RAG (Prompting)

Sos un asistente experto de RRHH. Respondé usando SOLO este contexto.
Si no sabés la respuesta, decí exactamente 'Información insuficiente'. No inventes.

Contexto recuperado:

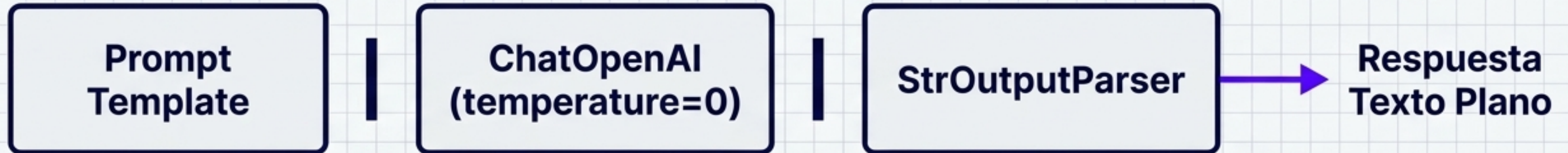
{context}

Pregunta del usuario: {question}

```
from langchain_core.prompts import ChatPromptTemplate  
prompt = ChatPromptTemplate.from_template(template)
```

Regla de Grounding: Le quitamos al modelo el permiso de usar su conocimiento previo. Todo se basa en la evidencia inyectada.

Paso 7 (C): LLM y LCEL



```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(temperature=0)
context_text = '\n\n'.join([d.page_content for d in docs])

chain = prompt | llm | StrOutputParser()
respuesta = chain.invoke({'context': context_text, 'question': user_q})
```

Temperatura 0: Buscamos hechos fríos y consistentes, no creatividad. Extraemos texto plano para armar el JSON nosotros.

Paso 7 (D): Construcción del Contrato JSON

- ✓ user_question
- ✓ system_answer
- ✓ chunks_related (content + id)

Cero claves extra permitidas.

```
import json

output = {
    'user_question': user_q,
    'system_answer': respuesta,
    'chunks_related': [
        {'chunk_id': f'chunk_{i}', 'content': d.page_content}
        for i, d in enumerate(docs)
    ]
}

print(json.dumps(output, indent=2, ensure_ascii=False))
```

Por qué Python: No le pedimos el JSON al LLM en el prompt porque es propenso a fallar. Al armarlo en Python, aseguramos un 100% de precisión estructural.

Paso 9: Validación Automática con Pytest

```
pytest tests/
```

```
===== 1 passed in 0.02s =====
```

```
def test_schema_keys():  
    # Simulamos el output  
    mock_json = {  
        'user_question': 'q', 'system_answer': 'a', 'chunks_related': []  
    }  
    # Verificamos contrato estricto  
    assert 'user_question' in mock_json  
    assert 'system_answer' in mock_json  
    assert 'chunks_related' in mock_json  
    assert len(mock_json.keys()) == 3 # Ninguna clave extra
```

Protección del Contrato: El test no mide la inteligencia de OpenAI. Mide que ningún desarrollador agregue accidentalmente una clave extra que repruebe el PI.

Paso 8: Generando outputs de ejemplo

outputs/sample_queries.json file

1. Factual

¿Cuál es el rol de un admin?

Dato duro. El bot responde.

2. Procedimiento

¿Cómo importo empleados por CSV?

Secuencia de pasos.

3. Sin Evidencia

¿Cómo instalo un servidor Linux?

El bot DEBE abstenerse.
Prueba el grounding.

```
python src/query.py --q 'Pregunta factual' >> outputs/sample_queries.json
```

Requisito de la Rúbrica: El evaluador revisará este archivo para comprobar que el sistema defensivo del bot funciona y no alucina.

Actividad Central: Diagnóstico RAG en Vivo

Tipo de Consulta	1. ¿Chunks útiles? (Retriever)	2. ¿Respondió con evidencia? (LLM)	3. ¿JSON Válido?
Factual	☐ Sí / ☐ No	☐ Sí / ☐ No	☐ Sí / ☐ No
Procedimiento	☐ Sí / ☐ No	☐ Sí / ☐ No	☐ Sí / ☐ No
Sin Evidencia	☐ Irrelevantes	☐ Se abstuvo	☐ Sí / ☐ No

El Corazón del Debug

Cuando el modelo alucina, NO miren al LLM primero. Revisen el **retriever**. Si entra basura, sale basura.

Pregunta Crítica: ¿Qué mejorarían primero? ¿El **Chunking**, el **Prompt**, o el **Top-K**?

Checklist de Entrega del Proyecto Integrador

Cimientos e Índice

- ✓ `faq_document.txt` (≥ 1000 palabras)
- ✓ `build_index.py` genera ≥ 20 chunks limpios
- ✓ Base Chroma en `storage/chroma/`

Consulta y Código

- ✓ `query.py` valida entradas vacías
- ✓ Retriever usa Top-K (2 a 5 chunks)
- ✓ `.env.example`, `.gitignore`, `requirements.txt`

Contrato Final (JSON)

- ✓ 3 claves exactas (`user_question`, `system_answer`, `chunks_related`)
- ✓ Cero claves extra
- ✓ 3 ejemplos en `outputs/` y test `pytest`

Repo Listo

Índice **OK**

Query **OK**

PI Aprobado